WEEK 2 – MACHINE LEARNING & AI

Solved Assignments & Mini Project: Supervised Learning (Regression & Classification)

Source basis: the uploaded Week 2 sheet covers Linear Regression, Logistic Regression, Decision Tree/Random Forest, and metrics including MSE, R² and Confusion Matrix. It assigns (1) a Linear Regression housing-price task and (2) Logistic Regression on Titanic survival, plus Mini Project 2: House Price Prediction Model using the Kaggle House Prices Dataset. ■filecite■turn0file0■L5-L19■

The third page specifies the mini-project tasks as training a Linear Regression model, predicting house prices, evaluating R², and plotting predicted versus actual values. ■filecite■turn0file0■L22-L45■

Important: the uploaded sheet does not contain the actual housing/Titanic CSV files, so this solution provides complete runnable Python programs and explains where the real dataset files should be supplied. Numerical scores are therefore generated when the programs are executed on the datasets.

1. Assignment 1 — Linear Regression on Housing Dataset

Objective: Build a Linear Regression model to predict house price. Linear Regression learns a relationship between input features X and a continuous target y.

Basic equation: y = β■ + β■x■ + β■x■ + … + β■x■. The model chooses coefficients that minimize the sum of squared prediction errors.

Expected workflow: load data → inspect/clean data → select features and target → handle missing values → split into train/test sets → train Linear Regression → predict → calculate MSE and R² → inspect predictions.

Complete solution (housing.csv):

```
# Assignment 1: Linear Regression on a housing dataset

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load the dataset
df = pd.read_csv("housing.csv")

print("Shape:", df.shape)
print(df.head())
print(df.info())

# 2. Set the target column.
# Change "price" if your dataset uses another name such as SalePrice.
target = "price"

X = df.drop(columns=[target])
y = df[target]

# 3. Identify numerical and categorical columns
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object", "category", "bool"]).columns

# 4. Preprocessing
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
```

```
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

# 5. Create the model pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# 6. Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# 7. Train
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Evaluate
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error (MSE):", mse)
print("R² Score:", r2)

# 10. Show a few predictions
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
print("\nSample predictions:")
print(results.head(10))
```

Interpretation: MSE measures the average squared difference between actual and predicted prices; lower is better. R^2 measures the proportion of variance in the target explained by the model; values closer to 1 indicate a better fit, although performance depends on the dataset.

2. Assignment 2 — Logistic Regression on Titanic Dataset

Objective: predict whether a passenger survived (binary classification). The target is normally 0 = did not survive and 1 = survived.

Logistic Regression converts a linear score into a probability using the sigmoid function: σ(z) = 1 / (1 + e■■). A probability ≥ 0.5 is commonly classified as class 1.

Complete solution (titanic.csv):

```
# Assignment 2: Logistic Regression on Titanic survival

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# 1. Load Titanic data
df = pd.read_csv("titanic.csv")

print("Shape:", df.shape)
print(df.head())

# 2. Select useful features.
# These names match the common Kaggle Titanic dataset.
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
target = "Survived"

X = df[features]
y = df[target]
```

```
# 3. Feature groups
numeric_features = ["Pclass", "Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Sex", "Embarked"]

# 4. Preprocessing
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

# 5. Logistic Regression pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

# 6. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 7. Train
model.fit(X_train, y_train)

# 8. Predict
y_pred = model.predict(X_test)

# 9. Evaluate
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix interpretation:

	Predicted 0	Predicted 1
Actual 0	True Negative (TN)	False Positive (FP)
Actual 1	False Negative (FN)	True Positive (TP)

Accuracy = (TP + TN) / (TP + TN + FP + FN). For imbalanced datasets, also inspect precision, recall and F1-score instead of relying only on accuracy.

3. Mini Project 2 — House Price Prediction Model

Dataset: Kaggle – House Prices Dataset. The uploaded assignment specifically names this dataset and asks for Linear Regression, house-price prediction, R^2 evaluation and an actual-versus-predicted plot. ■filecite■turn0file0■L31-L45■

For the standard Kaggle House Prices dataset, the target column is typically SalePrice. The code below is designed for train.csv from that dataset.

Complete mini-project code:

```
# Mini Project 2: House Price Prediction Model
# Dataset: Kaggle House Prices - train.csv

import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# ------------------------------------------------------------
# 1. Load data
# ------------------------------------------------------------
df = pd.read_csv("train.csv")

print("Dataset shape:", df.shape)
print("\nFirst five rows:")
print(df.head())

# ------------------------------------------------------------
# 2. Define target and features
# ------------------------------------------------------------
target = "SalePrice"

X = df.drop(columns=[target])
y = df[target]

# ID is only an identifier, so remove it if present.
if "Id" in X.columns:
    X = X.drop(columns=["Id"])

# ------------------------------------------------------------
# 3. Separate numerical and categorical features
# ------------------------------------------------------------
numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

# ------------------------------------------------------------
# 4. Preprocessing
# ------------------------------------------------------------
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

# ------------------------------------------------------------
# 5. Build Linear Regression pipeline
# ------------------------------------------------------------
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# ------------------------------------------------------------
# 6. Train/test split
# ------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# ------------------------------------------------------------
# 7. Train the model
# ------------------------------------------------------------
model.fit(X_train, y_train)

# ------------------------------------------------------------
# 8. Predict house prices
# ------------------------------------------------------------
y_pred = model.predict(X_test)
```

```
# ---------------------------------------------------------
# 9. Evaluate using MSE and R²
# ---------------------------------------------------------
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n===== MODEL PERFORMANCE =====")
print("Mean Squared Error:", mse)
print("R² Score:", r2)

# ---------------------------------------------------------
# 10. Predicted vs actual table
# ---------------------------------------------------------
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print("\nSample predictions:")
print(comparison.head(10))

# ---------------------------------------------------------
# 11. Plot predicted vs actual values
# ---------------------------------------------------------
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.6)

# Ideal prediction line
minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())
plt.plot([minimum, maximum], [minimum, maximum], linestyle="--")

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices")
plt.tight_layout()
plt.show()
```

4. How the Mini Project Works

Step 1 — Load: train.csv is read using pandas. **Step 2 — Target:** SalePrice is separated from the input features. **Step 3 — Cleaning:** missing numerical values are replaced with medians and missing categorical values with the most frequent category. **Step 4 — Encoding:** categorical variables are converted into numerical one-hot features. **Step 5 — Split:** 80% is used for training and 20% for testing. **Step 6 — Training:** Linear Regression learns coefficients from the training set. **Step 7 — Prediction:** the trained model predicts prices for unseen test rows. **Step 8 — Evaluation:** MSE and R² are calculated. **Step 9 — Visualization:** predicted prices are plotted against actual prices.

5. Metrics — Quick Revision

Metric	Formula / Meaning	Better value
MSE	Mean of (actual – predicted)²	Lower
R²	1 – SSres/SStot; variance explained by model	Closer to 1
Accuracy	(TP + TN) / Total	Higher

6. Installation & Execution

```
# Create/activate a virtual environment if desired, then install:

pip install pandas numpy scikit-learn matplotlib

# Run:
python linear_regression_housing.py
python titanic_logistic_regression.py
python house_price_prediction.py
```

Recommended project folder:

```
week2_ml/
```

```
■
■■■ linear_regression_housing.py
■■■ titanic_logistic_regression.py
■■■ house_price_prediction.py
■■■ housing.csv
■■■ titanic.csv
■■■ train.csv
```

7. Expected Output

Because the uploaded Week 2 sheet contains the assignment descriptions but not the actual CSV datasets, the exact numerical MSE, R^2 and accuracy values cannot be truthfully stated from the source alone. When the programs are run with the corresponding datasets, they print the metrics and display the required predicted-versus-actual graph.

8. Viva / Interview Questions

Q1. What is supervised learning? Learning from labeled input-output examples to predict outputs for unseen inputs.
Q2. Regression vs classification? Regression predicts continuous values such as price; classification predicts categories such as survived/not survived.
Q3. Why split into train and test? To evaluate how well the model generalizes to unseen data.
Q4. Why handle missing values? Many ML algorithms cannot directly train on missing values, and careless handling can reduce model quality.
Q5. Why one-hot encode categorical features? Linear models require numerical inputs, so categories are represented as binary indicator columns.
Q6. What does R^2 mean? It measures how much of the variation in the target is explained by the regression model relative to a baseline using the target mean.
Q7. What does MSE mean? It is the average squared prediction error; large errors receive greater penalty.
Q8. What is a confusion matrix? A table showing true/false positive and negative classification outcomes.
Q9. Why use a pipeline? It keeps preprocessing and modeling together and helps prevent inconsistent transformations between training and testing data.
Q10. What is overfitting? When a model learns the training data too closely and performs poorly on unseen data.

9. Conclusion

The Week 2 work demonstrates the complete supervised-learning workflow: preprocessing, train/test splitting, model training, prediction and evaluation. The two assignments cover regression and binary classification, while Mini Project 2 applies Linear Regression to house-price prediction and visualizes predicted versus actual prices, matching the tasks in the uploaded Week 2 sheet. ■filecite■turn0file0■L23-L45■